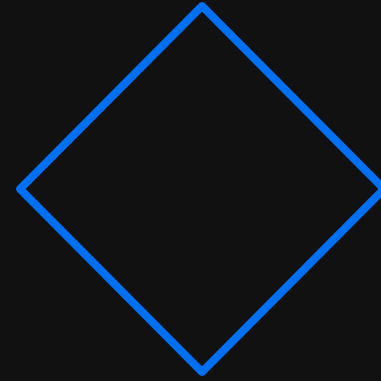


Next.js en détail



Séance 11 - Histoire, architecture (front <-> back), et Prisma/Neon dans Next

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

genese - rendu serveur & hydratation - frontière server/client - Prisma singleton - serverless

Où on en est, où on va

On approfondit Next.js

Acquis (S10)

- Next = React + backend, un seul projet
- Server vs Client Components (l'idée)
- Routing par fichiers
- Route Handlers (l'API sans Express)
- On a créé une première app Next

Objectifs de la séance

- L'HISTOIRE de Next (d'où ça vient, pourquoi)
- TECHNIQUE : comment front et back sont connectés
- Rendu serveur, hydratation, frontière server/client
- Route Handlers vs Server Actions
- Prisma + Neon DANS Next (ce qui est NOUVEAU)

Cap de la séance. La S10 était un survol. Aujourd'hui on OUVRE LE CAPOT : comment Next fabrique une page, comment le code du serveur et celui du navigateur se rencontrent, et comment on branche Prisma/Neon proprement. Pour la partie base de données, on ne réexplique PAS Prisma - on voit seulement ce qui est SPECIFIQUE à Next.

PARTIE 1

Histoire & philosophie de Next.js

La genèse : pourquoi Next existe

2016 - le problème que React seul ne réglait pas

En 2016, React est partout, mais il a un défaut : il rend TOUT dans le navigateur. La page arrive vide, puis le JavaScript la remplit. Consequences : mauvais référencement (Google voit une page vide) et premier affichage lent. Faire du rendu cote serveur avec React etait possible... mais tres penible a configurer.

La solution : Next.js

- Cree par Vercel (ex-ZEIT),
- par Guillermo Rauch - oct. 2016
- Rendu cote serveur PRET a l'emploi
- Routing par fichiers des le depart
- Zero config pour demarrer

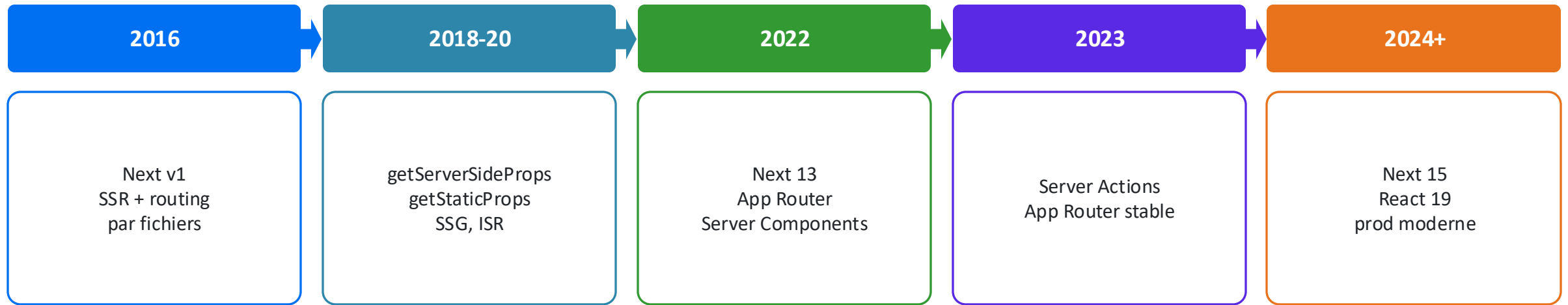
La philosophie

- Le meilleur de React, sans la douleur
- Convention plutot que configuration
- Optimise par default (vitesse, SEO)
- Un framework, pas juste une librairie
- 'React pour la production'

Idee-cle : Next ne remplace pas React, il l'ENCADRE pour resoudre ses angles morts (rendu serveur, routing, performance).

L'évolution en quelques étapes

De Pages Router à App Router



- **Deux 'eras'** : l'ancienne (Pages Router, `getServerSideProps`) et la moderne (App Router).
- **Le grand tournant = 2022 (Next 13)** : les React Server Components et le dossier `app/`.
- **Ce qu'on apprend = la version MODERNE** (App Router). Vous verrez parfois de vieux tutos avec `pages/`.

Les technologies actuelles

Où en est Next.js en 2025-2026

Amenons la timeline jusqu'à aujourd'hui. En 2025-2026, plusieurs choses se sont stabilisées et sont devenues la norme.

React 19

- Les Actions (Server Actions)
- `useActionState`, `useOptimistic`
- le hook `use`
- -> moins de `useState/useEffect`

Turbopack

- Le nouveau compilateur, STABLE
- Démarrage plus rapide
- Hot-reload quasi instantané
- -> meilleure expérience dev

Rendu (PPR)

- Partial Prerendering
- statique instantané +
- zones dynamiques
- -> le meilleur des deux mondes

Bonne nouvelle. App Router, Server/Client Components, Server Actions : c'est exactement la DIRECTION que prend l'écosystème. Vous apprenez le STANDARD ACTUEL - pas quelque chose qui va vieillir.

L'ère de l'IA

L'IA change le dev web sur deux plans

1) L'IA pour ECRIRE le code

- Copilot, Cursor, Claude Code
- v0 de Vercel : genere des
- composants / interfaces
- a partir d'une phrase
- -> le code va plus vite

2) L'IA DANS vos apps

- Chatbot, resume, recherche
- Vercel AI SDK (paquet 'ai')
- streaming des reponses (mot a mot)
- appelle COTE SERVEUR
- (Route Handler / Server Action)

Lien avec la seance : la CLE d'API du modele est un SECRET -> 'server-only', comme Prisma. Jamais dans un 'use client', jamais dans une variable NEXT_PUBLIC_.

L'IA remplace la frappe au clavier, pas la COMPREHENSION.

A l'ere de l'IA, les fondamentaux comptent PLUS, pas moins : il faut savoir lire, verifier, securiser.

L'apport de Next : les modes de rendu

Où et quand la page est fabriquée

Mode	Où / quand la page est fabriquée	Bon pour
CSR (React classique)	dans le navigateur, après coup	apps privées (dashboard)
SSR (server-side)	sur le serveur, à chaque requête	contenu dynamique + SEO
SSG (static)	à la compilation, une fois	pages qui changent peu (blog)
ISR (incremental)	static + régénérer périodiquement	gros sites, cache malin

React seul ne faisait que du CSR (page vide au départ). Next apporte SSR, SSG et ISR : c'est SA grande valeur ajoutée. Avec l'App Router, le rendu serveur devient même le comportement par DÉFAUT.

PARTIE 2

Comment le front et le back se connectent

Un serveur, deux mondes

Le modèle mental de Next

Le monde SERVEUR

- Tourne sur le serveur (Node)
- Accès BD, secrets, systeme de fichiers
- Genere le HTML de depart
- Le code n'est PAS envoye au navigateur
- = Server Components, Route Handlers, Actions

Le monde NAVIGATEUR

- Tourne dans le navigateur
- Interactivite : clics, saisies, etat
- useState, useEffect, onClick
- Recoit le HTML puis 'prend vie'
- = Client Components ('use client')

La grande difference avec avant : ce n'etait pas 'deux projets' (front + back) qui se parlent par HTTP. C'est UN SEUL projet ou une partie du code s'execute sur le serveur et une autre dans le navigateur. Next gere la couture entre les deux. Tout l'enjeu technique de la seance : comprendre cette COUTURE.

Que se passe-t-il quand on ouvre une page ?

Rendu serveur -> HTML -> hydratation



L'HYDRATATION, c'est le mot cle :

- Le serveur envoie d'abord une page **HTML complete** -> affichage immediat + bon SEO.
- Puis le **JavaScript** arrive et '**hydrate**' la page : il rebranche les boutons, les etats, les evenements.
- **Avant hydratation** : on VOIT la page mais elle n'est pas encore cliquable.
- C'est pour ca qu'un **Client Component** est rendu **2 fois** : une fois sur le serveur (HTML), une fois dans le navigateur (vie).

La frontière serveur / client

Ce qui traverse le réseau (et ce qui ne peut pas)

```
// Server Component (parent)
import Bouton from './Bouton' // Client

export default async function Page() {
  const data = await prisma.pokemon.findMany()
  // on passe des DONNEES (serialisables) a un Client
  return <Bouton pokemons={data} />
}
```

Next

La regle de la frontiere

- Un Server passe des PROPS a un Client
- Ces props traversent le RESEAU
- -> elles doivent etre SERIALISABLES
- OK : objets, tableaux, string, nombre
- PAS OK : une fonction, une connexion,
- le client Prisma lui-meme
- (sauf les Server Actions - cas special)

Autrement dit : le serveur fait le travail 'lourd' (lire la BD), puis n'envoie au navigateur que le RESULTAT (des donnees simples). Le navigateur ne voit jamais Prisma ni vos secrets - il recoit juste des Pokemon deja prêts.

Server Components en détail

Le défaut : puissant, mais sans interactivité

Ce qu'ils PEUVENT

- Etre async (await direct)
- Lire la BD (Prisma), lire des fichiers
- Utiliser des secrets (.env) sans risque
- Appeler d'autres services
- -> rendus UNE fois, sur le serveur

Ce qu'ils NE PEUVENT PAS

- useState / useEffect
- onClick et autres evenements
- Acceder a window / document
- Etre interactifs
- -> pour ca : un Client Component

Leur super-pouvoir : leur code reste sur le serveur, il n'est JAMAIS envoye au navigateur. Donc on peut y mettre une requete Prisma ou une cle secrete sans crainte. C'est la premiere grande difference avec le React que vous connaissez.

Client Components en détail

L'interactivité, hydratée dans le navigateur

```
"use client" // la directive frontiere
import { useState } from "react"

export default function Bouton({ pokemons }) {
  const [i, setI] = useState(0)
  return <button onClick={() => setI(i + 1)}>...</button>
}
```

Next

À savoir

- "use client" en HAUT du fichier
- = ce fichier (et ses enfants) au client
- Rendu 2x : HTML (serveur) puis hydrate
- Peut RECEVOIR des données d'un Server
- Recoit son code dans le navigateur

'use client' n'est PAS 'execute uniquement dans le navigateur'. C'est une FRONTIERE : a partir de cette ligne, le composant est aussi envoyé au navigateur pour devenir interactif. Règle pratique : gardez les Client Components petits et 'aux feuilles' de l'arbre (les boutons, les formulaires), et laissez les parents en Server.

Le 'backend' dans Next : 3 outils

Selon qu'on LIT ou qu'on ECRIT

Server Component

- POUR : lire et afficher
- au chargement d'une page
-
- await prisma...findMany()
- directement dans la page
-
- -> le plus simple pour LIRE

Route Handler

- POUR : une vraie API HTTP
- (app/api/.../route.ts)
-
- Appelable par n'importe qui
- (un autre front, une appli mobile)
-
- -> comme vos routes Express

Server Action

- POUR : écrire/modifier
- depuis un formulaire ou un clic
-
- Fonction serveur appelée
- directement depuis le client
-
- -> pas besoin d'écrire d'API

Règle simple : LIRE au chargement -> Server Component. Exposer une API réutilisable -> Route Handler. ECRIRE depuis un bouton/formulaire -> Server Action (le plus moderne).

Server Actions : muter sans écrire d'API

Une fonction serveur appelée depuis le client

```
// app/actions.ts
"use server" // <- fonctions serveur
import { prisma } from "@lib/prisma"
import { revalidatePath } from "next/cache"

export async function capturer(nom: string) {
  await prisma.pokemon.create({ data: { nom } as any })
  revalidatePath("/pokedex") // rafraichit la page
}

// dans un Client Component : <button onClick={() => capturer("pikachu")}>
```

Next

L'idée

- "use server" = fonction
- qui tourne sur le serveur
-
- Appelable directement
- depuis le navigateur
- (Next fait la requete pour vous)
-
- revalidatePath : recharge
- les donnees affectees
-
- -> plus besoin d'axios pour
- les mutations simples

PARTIE 3

Prisma + Neon dans Next.js

Prisma : ce qu'on NE refait pas

On ne réexplique pas les bases

Deja vu (identique) - on saute

- schema.prisma (modeles, enums, relations)
- npx prisma migrate dev / generate
- findMany / create / update / delete
- prisma studio
- Neon = PostgreSQL heberge
- -> AUCUN changement ici

NOUVEAU (specifique a Next)

- OU mettre le client (lib/prisma.ts)
- Le piege du hot-reload (singleton)
- Prisma est 'server-only'
- OU appeler Prisma (Server / Handler / Action)
- Neon en environnement serverless
- -> c'est CA qu'on voit aujourd'hui

Le message : votre savoir Prisma est INTACT. Ce qui change, ce n'est pas Prisma - c'est l'ENVIRONNEMENT (Next, serverless) dans lequel on l'utilise, et quelques regles a respecter.

Le piège n°1 : le hot-reload

Pourquoi le singleton globalThis

En dev, Next recharge le code a chaque sauvegarde. Si on écrit 'new PrismaClient()' naïvement, on en crée une NOUVELLE a chaque rechargement -> des dizaines de connexions -> 'too many connections'. Solution : stocker l'instance sur globalThis.

```
// lib/prisma.ts
import { PrismaClient } from "@generated/prisma/client"
import { PrismaNeon } from "@prisma/adapters-neon"

const adapter = new PrismaNeon({ connectionString: process.env.DATABASE_URL })

const g = globalThis as unknown as { prisma?: PrismaClient }
export const prisma = g.prisma ?? new PrismaClient({ adapter })
if (process.env.NODE_ENV !== "production") g.prisma = prisma
```

Next

On REUTILISE l'instance rangée sur globalThis si elle existe déjà ; on n'en crée une neuve que la première fois. En prod, pas de hot-reload, donc pas de problème.

Le piège n°2 : Prisma est 'server-only'

Jamais côté navigateur

Pourquoi

- Prisma parle à la BD
- et utilise vos secrets (DATABASE_URL)
- Si ce code partait au navigateur :
- fuite de secrets = catastrophe
- En plus, ça ne marcherait pas

Où l'utiliser (uniquement)

- Server Components
- Route Handlers (route.ts)
- Server Actions ('use server')
- = le monde SERVEUR
- JAMAIS dans un 'use client'

```
// (optionnel) forcer 1 erreur si importe côté client :  
import "server-only"  
// -> le build échoue si un Client Component importe ce fichier
```

[Next](#)

Règle d'or : le fichier lib/prisma.ts ne doit être importé que par du code serveur.

Lire la BD : dans un Server Component

Le cas le plus courant

```
// app/pokedex/page.tsx (Server Component)
import { prisma } from "@/lib/prisma"

export default async function Page() {
  const pokemons = await prisma.pokemon.findMany({
    orderBy: { numeroPokedex: "asc" },
  })
  return <ul>{pokemons.map(p =>
    <li key={p.id}>{p.nom}</li>)}</ul>
}
```

Next

Ce qui est neuf

- La page est async
- On importe le singleton
- prisma.pokemon.findMany()
- DIRECTEMENT dans la page
-
- Pas de useEffect,
- pas d'axios, pas d'API
- -> la donnée arrive prête

La requête Prisma s'exécute sur le SERVEUR ; le navigateur ne reçoit que le HTML final. C'est ça, le confort de Next côté lecture.

Écrire : Route Handler ou Server Action

Deux façons de créer/modifier

Option A - Route Handler (API) :

```
// app/api/pokedex/route.ts
import { prisma } from "@lib/prisma"

export async function POST(req: Request) {
  const body = await req.json()
  const p = await prisma.pokemon
    .create({ data: body })
  return Response.json(p, { status: 201 })
}
```

Next

Option B - Server Action :

```
"use server"
import { prisma } from "@lib/prisma"
import { revalidatePath } from "next/cache"

export async function capturer(nom: string) {
  await prisma.pokemon.create({ data:{nom} as any })
  revalidatePath("/pokedex")
}
```

Next

- **Route Handler** : une vraie URL /api/... - utile si d'autres clients l'appellent.
- **Server Action** : appelée directement depuis un bouton/formulaire, sans écrire d'API.
- **Dans les deux cas** : Prisma tourne sur le serveur, en sécurité.

Neon + Next : le contexte serverless

Pourquoi l'adaptateur Neon compte

En production (ex. Vercel), une app Next tourne souvent en 'serverless' : de petites fonctions demarreées a la demande, tres nombreuses. Probleme : chacune voudrait ouvrir sa connexion a la BD -> on epuise vite les connexions Postgres.

Les solutions

- Connexion POOLED de Neon (PgBouncer)
- OU le pilote serverless Neon (HTTP)
- + l'adaptateur @prisma/adaptateur-neon
- -> celui que vous utilisez DEJA !
- = Prisma compatible serverless/edge

Variables d'environnement

- DATABASE_URL : cote SERVEUR seulement
- jamais exposee au navigateur
- Prefixe NEXT_PUBLIC_ = visible cote client
- -> JAMAIS pour un secret / la BD
- Sur Vercel : a configurer dans le dashboard

Bonne nouvelle : l'adaptateur Neon (@prisma/adaptateur-neon) que vous connaissez deja est justement ce qui rend Prisma heureux en serverless. Vous etes deja bien equipes.

Mise en place dans un projet Next

Les étapes concrètes

```
# 1) dans le projet Next
npm install @prisma/client @prisma/adapters-neon
npm install -D prisma
npx prisma init                # cree prisma/ + .env
# 2) schema.prisma (vous savez deja) -> npx prisma migrate dev
# 3) creer lib/prisma.ts (le singleton, slide 17)
# 4) l'importer dans un Server Component / Route Handler / Action
```

bash

- **La difference vs le cours Prisma initial** : le fichier lib/prisma.ts (singleton) et OU on l'appelle.
- **On garde le meme schema**, la meme base Neon, les memes migrations.
- **On peut meme REUTILISER le schema du Pokedex** existant : rien a re-modeliser.
- **Prochaine seance** : brancher tout ca dans le Pokedex version Next.

Récapitulatif de la séance

Ce qu'il faut retenir

Histoire

- Ne par Vercel en 2016
- Pour le rendu serveur de React
- Tournant 2022 : App Router
- + React Server Components
- Modes : CSR/SSR/SSG/ISR

Front <-> Back

- Un serveur, deux mondes
- HTML rendu serveur + hydratation
- Frontière : props serialisables
- Server / Client Components
- Route Handlers / Server Actions

Prisma dans Next

- lib/prisma.ts (singleton)
- globalThis (anti hot-reload)
- server-only (jamais au client)
- lire = Server Component
- Neon adapter = serverless OK

Le fichier à retenir :

```
// lib/prisma.ts : un seul client, range sur globalThis, avec l'adaptateur Neon
// importe UNIQUEMENT cote serveur (Server Component / route.ts / 'use server')
```

Next



Questions ?

Prochaine étape : Prisma/Neon branche dans le Pokedex version Next.

Le capot est ouvert. Vous savez comment Next fonctionne.

